HEALTHCARE ANALYTICS METHODOLOGY

Analyst: Tessy Odiley

OCTOBER 28, 2026
ANALYSTLAB AFRICA

HEALTHCARE ANALYTICS METHODOLOGY

1. PROJECT OVERVIEW

Introduction

This project aims to develop a Healthcare Intelligence Dashboard using MySQL and Power BI to analyze healthcare activity, financial performance, procedure utilization, operational efficiency, and patient return behavior. The objective was to transform raw healthcare data into actionable insights that can support evidence-based decision-making and improve operational visibility.

The project utilized five interconnected datasets containing encounter, patient, procedure, payer, and organization information. A structured data quality assessment was conducted to validate the accuracy, completeness, and integrity of the data prior to analysis. The datasets were then analyzed using SQL and visualized through a series of interactive Power BI dashboards.

The final solution provides stakeholders with a consolidated view of healthcare operations through five analytical dashboards: Executive Overview, Encounters, Procedures and Base Cost, Operations, and Patient Returns Analyses.

Business Problem

The healthcare organization has provided historical data from 2011 to 2022 for analysis because management wants to understand:

- Encounter volumes and costs
- Procedure utilization
- Payer coverage
- Hospital Operations

Business Questions

Question	Relevant Metric	Calculation (DAX Measures)	Interpretation

How have encounter volumes changed over time?	Total Encounters vs Date	- **Total Encounters =**
		`DISTINCTCOUNT('hospital_analytics encounters'[encounter_id])`
		- The Date Table was created using:
		`DateTable = VAR MinYear = YEAR(MIN('hospital_analytics encounters'[start_date])) VAR MaxYear = YEAR(MAX('hospital_analytics encounters'[stop_date])) RETURN ADDCOLUMNS (CALENDAR (DATE(MinYear, 1, 1), DATE(MaxYear, 12, 31)), "Year", YEAR ([Date]), "Month", FORMAT ([Date], "mmm"), "Month Number", MONTH ([Date]), "Month Year", FORMAT ([Date], "MMM YYYY"), "YearMonth", YEAR ([Date]) * 100 + MONTH ([Date]), "Quarter", "Q" & FORMAT ([Date], "Q"), "Day", FORMAT ([Date], "ddd"), "Day Number", WEEKDAY ([Date]))`
			Over the analyzed period, total encounters remained relatively stable except for a spike in encounter numbers around January, 2014. This indicates that the hospital has a steady number of patients visiting. The spike in 2014 requires further investigation to understand the factors behind it.

Which encounter classes account for the largest proportion of recorded encounters?	Encounter class vs Total Encounters	- **Total Encounters =** `DISTINCTCOUNT('hospital_analytics encounters'[encounter_id])` - The encounter class was selected with the total encounters measure to create a donut chart showing the encounter proportions.	The ambulatory encounter class accounted for 44.95% of the total encounters that occurred during the reporting period. This indicates that almost 45% of patients visiting the hospital were ambulatory patients. Further analysis is required to find out what type of procedures were

			performed during their visits to the hospital.

What proportion of encounters lasted more than 24 hours, where valid start and end timestamps are available?	Encounters Longer than 24 Hours vs Percentage Over 24 Hours	- **Encounters Longer than 24 Hours =**
		`CALCULATE(`
		`COUNTROWS('hospital_analytics encounters'),`
		`'hospital_analytics encounters'[duration_hours] > 24`
		`)`
		- **Percentage Over 24 Hours =**
		`DIVIDE(`
		`[Encounters Longer than 24 Hours],`
		`[Total Encounters]`
		`)`
			75 Encounters were longer than 24 hours, accounting for 0.27% of total encounters. This indicated that most patient encounters short in duration. However, this small percentage of short encounters may demand many resources from the hospital, as they drive up the base costs incurred by the hospital.

How do total claim costs vary across encounter classes?	Encounter class vs Total Claim Costs	**Total Claim Cost** = `SUM('hospital_analytics encounters'[total_claim_cost])`	The ambulatory encounter class had the highest claim costs at $36M while wellness encounters accrued claim costs of $8M. The ambulatory encounter class accumulated the most hospital bills.
How much payer coverage is recorded?	Total Payer Coverage vs Coverage Percentage vs	- **Total Payer Coverage** = `SUM('hospital_analytics encounters'[payer_coverage])` - **Coverage Percentage** = `DIVIDE([Total Payer Coverage],[Total Claim Cost])`	Total payer coverage amounted to $31.1M, accounting for 30.6% coverage. This suggests that out of the $101,5M in total claim costs incurred by patients from encounters, only $31.1M was covered by insurance payments. Only 30.6% of claim costs were paid out by insurance. Further analysis is required to identify the factors impacting insurance payouts.

Which procedures occur most frequently?	Total Procedures vs Description	`Total Procedures = COUNT('hospital_analytics procedures'[patient_id])`	The "Assessment of health and social care needs" procedure was the most frequently occurring procedure during the analysis period. This shows that most patients who needed procedures had this procedure performed. More analysis required to determine whether this is a standalone procedure or if it is a standard procedure for most patients.
Which procedures have the highest average base cost, considering both frequency and the limitations of the data?	Average Procedure Base Cost vs Description	`Average Procedure Base Cost = AVERAGE('hospital_analytics procedures'[base_cost])`	The Electrical cardioversion procedure had the highest average base cost at $25.9K. This suggests that the hospital pays roughly $26K to keep beds open, staff available, and ensure continuous care for patients who undergo this procedure.
How do average total claim costs vary by payer?	Average Claim Costs vs Payer Name	`Average Claim Cost = AVERAGE('hospital_analytics encounters'[total_claim_cost])`	Medicaid members had the highest average claim costs at $6.2K, patients without insurance came in second with $5.6K, and Dual Eligible member had the lowest average claim costs of $1.7K. This shows that Medicaid members and patients with no insurance incurred the most hospital bills on average. Patients

			without insurance are accumulating high hospital bills. This may be risky place a financial burden on the hospital and the patients. Further analysis is required to determine how these payments are recovered by the hospital.
How many patients return for multiple encounters?	Total Patients vs Returning Patients vs Patient Return Rate	- **Total Patients** = DISTINCTCOUNT('hospital_analytics encounters'[patient_id]) - **Returning Patients** = `COUNTROWS( FILTER( VALUES('hospital_analytics encounters'[patient_id]), CALCULATE(COUNTROWS('hospital_analytics encounters')) > 1 ) )` - **Patient Return Rate** = DIVIDE([Returning Patients],[Total Patients])	During the reporting period, a 974 patients had encounters at the hospital. A total of 854 patients had more than one encounter at the hospital, resulting in a return rate of 87,7%. This suggests that the hospital has repeat patients who visit the hospital for more than one encounter. However, this doesn't mean translate into patient retention. Further analysis is required to investigate what procedures the patients are getting when the have more than one encounter in the hospital.

Dataset

The hospital provided five CSV files containing healthcare datasets that were integrated into a relational database structure.

Dataset	Description
Encounters	Contains encounter activity, timestamps, claim costs, payer coverage, and encounter classifications.
Patients	Contains demographic information including birth date, gender, race, ethnicity, and geographic location.
Procedures	Contains procedure descriptions, procedure timestamps, and base costs.
Payers	Contains healthcare payer and insurance provider information.
Organizations	Contains healthcare organization information including locations and identifiers.

Tools and Technologies

The following tools were used throughout the project:

Tool	Purpose
Excel	Data inspection, column header renaming, formatting, and storage
MySQL	Data cleaning, quality assessment, validation, and SQL analysis, aggregation, and metric verification
Power Query	Data transformation and preparation
Power BI Desktop	Data modeling, DAX calculations, dashboard development, and reporting

2. DATA PREPARATION AND QUALITY ASSESSMENT

Overview

Before analysis could be performed, the healthcare datasets were inspected and prepared to ensure accuracy, completeness, and suitability for analysis. In Excel, the dataset header columns were renamed appropriately, datetime columns were formatted to “yyyy-mm-dd hh:mm:ss”. Data preparation involved importing the datasets into MySQL, validating data types and formats, identifying quality issues (e.g., missing values, duplicates, misspellings, and inconsistent formatting), and performing corrective actions.

Data Import and Database Preparation

- A database named **hospital_analytics** was created in MySQL using:

Create new schema > name > apply > finish

- Separate tables were created for encounters, patients, procedures, payers, and organizations using the CREATE TABLE query, specifying the column header names, and data types stored in the tables, as shown in the picture below:

```
-- creating the payers table to store payer information
create table payers (
    payer_id varchar(255) NOT null primary key,
    name varchar(255),
    address varchar(255),
    city varchar(255),
    state_headquarters varchar(255),
    zip_code varchar(255),
    phone_number varchar(255)
);
```

This process was repeated for all the tables (see attached MySQL script document).

Appropriate data types were assigned during table creation to ensure data integrity and facilitate analysis. These included: VARCHAR data types for identifiers and descriptive attributes, DATE and DATETIME data types for date and time fields, and DECIMAL data types for financial measures such as claim costs, payer coverage, and procedure base costs.

- CSV files were imported using the load data local infile query, ensuring that the CSV column headers match the created table headers, as shown in below figure:

```
-- import patients table
LOAD data local infile 'C:/Users/totes/Downloads/patients.csv'
INTO table patients
FIELDS TERMINATED BY ','
optionally enclosed by '"'
LINES terminated by '\n'
ignore 1 rows;
```

Data Quality Checks

Following import, record and field counts were validated against the source files to confirm that all datasets had loaded successfully. Then, data quality checks were performed to identify missing values, duplicates, inconsistent formatting/data types, and datetime issues.

Total Record and Field Counts

After importing the data, the first data quality check involved counting the records and fields within each table and validating the result against the excel rows and columns to see if the data was imported correctly.

```sql
-- count the rows in each table
select count(*) AS record_count_encounters
from encounters;
select count(*) AS record_count_patients
from patients;
select count(*) AS record_count_organizations
from organizations;
select count(*) AS record_count_payers
from payers;
select count(*) AS record_count_procedures
from procedures;
```

```sql
-- count the columns for each table
select COUNT(*) AS column_count_encounters
from information_schema.columns
where table_schema = 'hospital_analytics'
and table_name = 'encounters';
```

The record count results are shown in the table below:

Table	Record Count	Field Count
Encounters	27,891	14
Payers	10	7
Procedures	47,701	9
Patients	974	20
Organizations	1	8

Missing Value Assessment

A missing value assessment was conducted to identify incomplete records and determine their potential impact on the analysis.

The SQL sum() query was used to evaluate null values and blank records across all datasets because MySQL records blanks as “1”, and the sum() function will aggregate all those ones to give an accurate count of the missing values. While importing the data from the CSV files, MySQL stored null records using blank quotation marks (‘’) so using the “is null” function returned an error. Instead, blank quotations were used in the query to search for missing values, as shown

below. The “is null” function was only used for the date time fields because MySQL imported those fields properly.

```
-- missing values in payers table
select
    sum(payer_name ='') as missing_name,
    sum(address ='') as missing_address,
    sum(city ='') as missing_city,
    sum(state_headquarters ='') as missing_state,
    sum(zip_code ='') as missing_zip_code,
     sum(phone_number ='') as missing_phone_number
    FROM payers;
```

Missing Values Summary

Table	Columns	Missing Value Count
Encounters	total_claim_cost	89
	payer_coverage	13,586
	reason_description and reason_code	19,541 each
Patients	death_date	820
	suffix	953
	marital_status	1
	zip_code	142
Procedures		
Payers	address	1
	city	1
	state	1
	zip_code	1
Organizations	N/A	N/A

Duplicate Record Assessment

Duplicate record checks were performed to ensure that each entity was uniquely represented within the database.

The following primary identifiers were used to perform the duplicate checks:

- encounter_id for encounters
- patient_id for patients
- payer_id for payers

- organization_id for organizations

```
-- check for duplicates in encounters tables
select encounter_id, count(*) as duplicate_encounters
from encounters
group by encounter_id
having count(*)>1;
```

The procedures table did not contain a single unique identifier and was therefore assessed using a combination of all the fields in the procedures table to identify potential duplicate records.

```
-- check for duplicates in procedures tables
select encounter_id, patient_id, start_date_time, stop_date_time, code, description, base_cost, count(*) as duplicate_procedures
from procedures
group by encounter_id, patient_id, start_date_time, stop_date_time, code, description, base_cost
having count(*)>1;
```

No duplicate records were identified within the datasets.

Data Type Validation

Data types were reviewed following import using the describe() function to verify that each field had been correctly interpreted by MySQL.

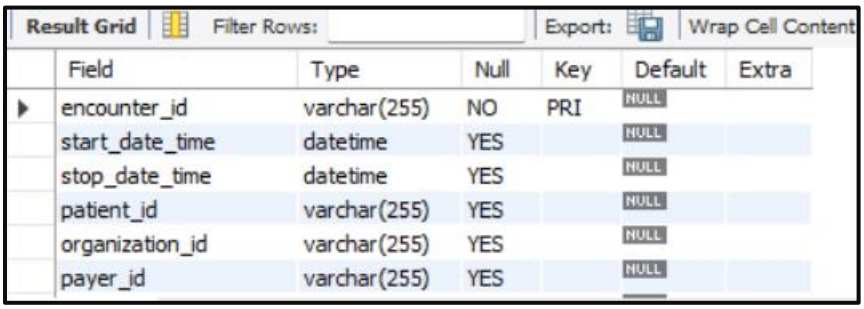
Particular attention was given to:

- Date fields
- Date-time fields
- Financial measures
- Identifier columns

```
-- check data types for the tables
describe encounters;
describe organizations;
describe patients;
describe payers;
describe procedures;
```

During this process, an issue was identified within the patient dataset where date fields were imported incorrectly. The CSV file date formats were corrected to the “yyyy-mm-dd” format, the patient table was recreated, and the data was re-imported into MySQL. Validation confirmed that

birth dates and death dates were subsequently imported correctly. The data types were correctly validated for each table, confirming correct import of the dataset.


Result Grid | Filter Rows: | Export: | Wrap Cell Content

Field	Type	Null	Key	Default	Extra
encounter_id	varchar(255)	NO	PRI	NULL	
start_date_time	datetime	YES		NULL	
stop_date_time	datetime	YES		NULL	
patient_id	varchar(255)	YES		NULL	
organization_id	varchar(255)	YES		NULL	
payer_id	varchar(255)	YES		NULL	

Date Validation

Date quality checks were performed to identify invalid or illogical date relationships.

The following validations were conducted:

- Verification that encounter and procedure end times occurred after start times.

```
-- date quality checks for invalid date order in procedures table
select *
from procedures
where stop_date_time < start_date_time;
```

- Verification that patient death dates did not occur before birth dates.

```
-- date quality checks for invalid date order in patients table
select *
from patients
where death_date < birth_date;
```

- Identification of future birth dates.

```
-- date quality checks for any birth_dates in the future
select *
from patients
where birth_date > current_date();
```

No significant date inconsistencies were identified after the data correction process. Encounter timestamps and patient date fields were considered suitable for analysis.

Defining Table Relationships

The relationships between tables were verified using the left join() query to ensure the tables can support accurate analytical modelling.

```
-- encounters to patients table
select count(*) as encounters_w_o_patients
from encounters e
left join patients p
    on e.patient_id = p.patient_id
where p.patient_id is null;
```

The following relationships were validated:

Primary Key	Foreign Key	Table Relationships
encounter_id	patient_id	Encounters > Patients
	organization_id	Encounters > Organizations
	payer_id	Encounters > Payer
N/A – the procedures table did not have a primary key	patient_id	Procedures > Patients
	encounter_id	Procedures > Encounters

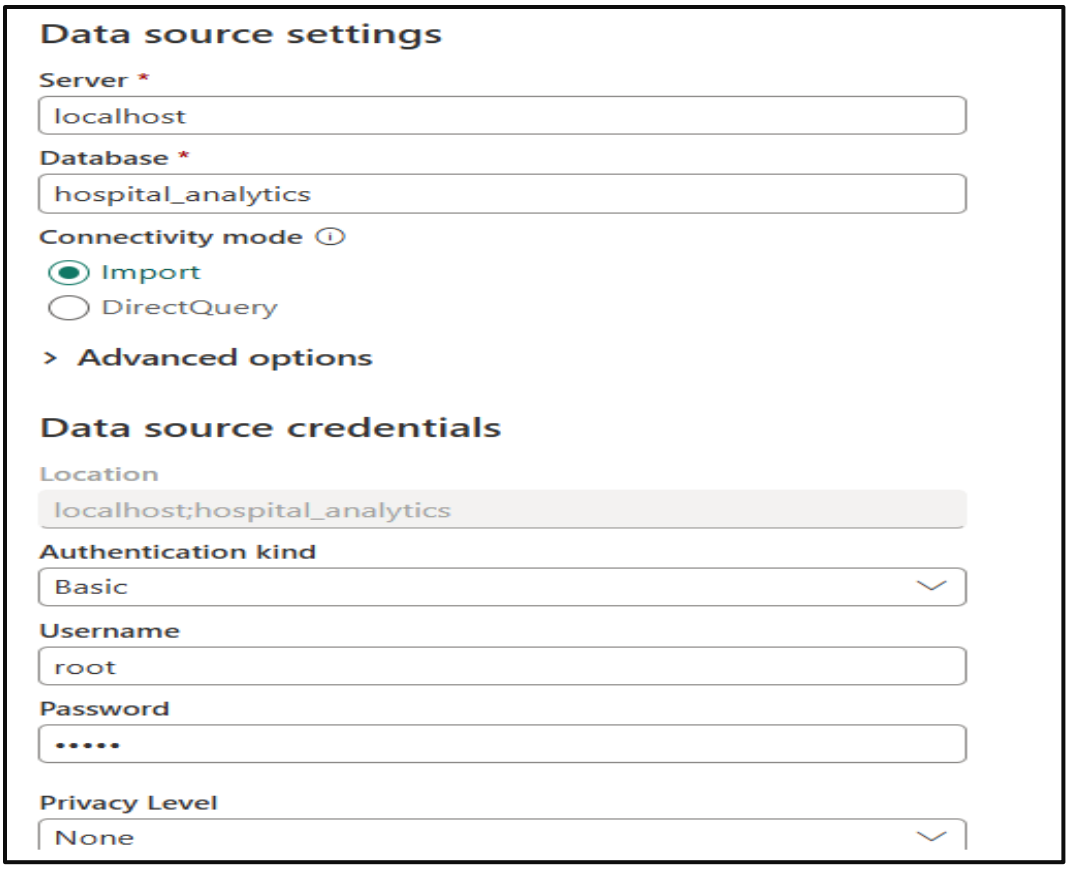
No unmatched records were found, indicating that all related records had corresponding entries in their parent tables. This confirmed that the data model was suitable for both SQL analysis and Power BI reporting.

3. DATA ANALYSIS IN POWER BI

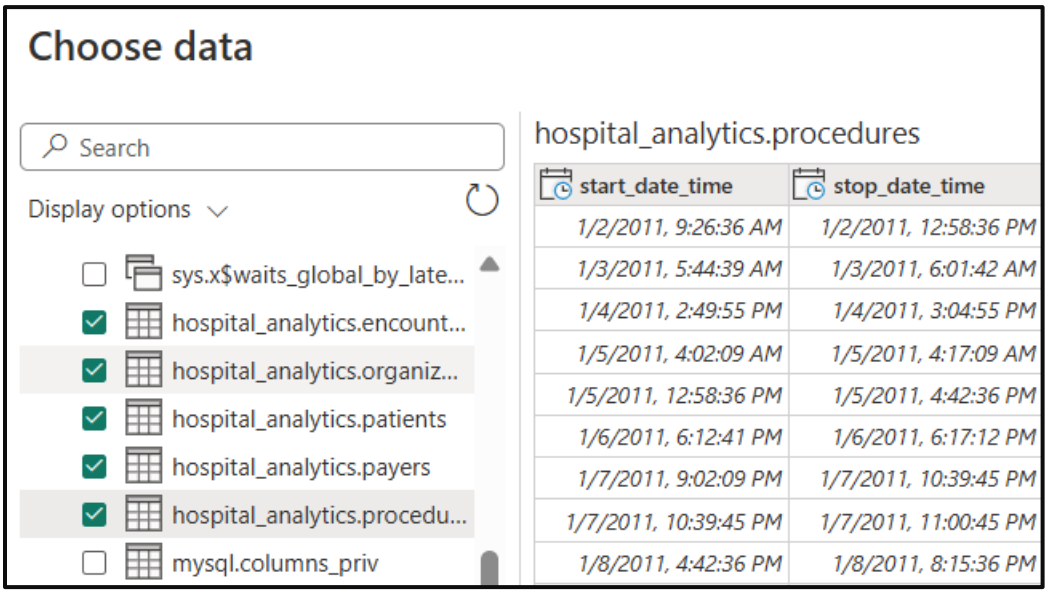
Data Extraction, Transformation and Loading in Power Query

- The dataset was extracted by connecting the database to Power BI using Power Query:
- **Get data > new sources > view more > database > MySQL database**

In the popup window, insert the appropriate server and database name:



- Check the tables from the dataset and click transform



Additional transformations were performed within Power Query to prepare the data for visualization and reporting.

These transformations included:

- Splitting encounter and procedure start and end timestamps into separate date and time fields.
- Cleaning and trimming text columns.
- Using “Custom Column” button and calculated formula to calculate the duration of encounters and procedures: **Add columns > custom column > duration_hours = Duration.Hours([stop_date_time]-[start_date_time]).**
- Creating encounter duration measures in hours.
- Standardizing data type formats across all imported tables.
- Replacing blank rows with “null” values.
- Removing irrelevant columns, such as zip_code, address, reason_code, and reason_description (too many null values).
- Patient first and last name columns were combined into a new column: patient_name.

These transformations ensured the data was adequately prepared and suitable for analysis.

DAX Modelling – STAR Schema

After transforming and loading the datasets, the table relationships were established in the DAX model view with the encounters table as the Facts table in the STAR schema. The table relationships were established using the primary and foreign keys, as shown in the table and picture below:



Date Table Creation

A dedicated Date table was created within Power BI to support time-based analysis and improve reporting consistency across all dashboard pages.

The Date table was generated using DAX and was configured to include the following attributes:

- Date
- Year
- Quarter
- Month
- Month Number – to sort the months correctly
- Month-Year
- Day
- Day Number – to sort the days correctly

The Date table was created using the following calculation:

```
DateTable =
VAR MinYear = YEAR( MIN( 'hospital_analytics encounters'[start_date]) )
VAR MaxYear = YEAR( MAX( 'hospital_analytics encounters'[stop_date]) )
RETURN
ADDCOLUMNS (
    CALENDAR (
        DATE( MinYear, 1, 1 ),
        DATE( MaxYear, 12, 31 )
    ),
    "Year", YEAR ( [Date] ),
    "Month", FORMAT ( [Date], "mmm" ),
    "Month Number", MONTH ( [Date] ),
    "Month Year", FORMAT ( [Date], "MMM YYYY" ),
    "YearMonth", YEAR ( [Date] ) * 100 + MONTH ( [Date] ),
    "Quarter", "Q" & FORMAT ( [Date], "Q" ),
    "Day", FORMAT ( [Date], "ddd" ),
    "Day Number", WEEKDAY ( [Date] )
)
```

A relationship was established between the date table and the encounters table using the start_date_time column. This relationship enabled the creation of accurate year-over-year, quarterly, monthly, and daily trend analyses.

The implementation of a dedicated Date table also supported the development of interactive date filters and ensured that all time-intelligence calculations used a consistent calendar structure.

Dashboard Development

Five dashboards were developed with corresponding KPIs and slicers: Executive Overview, Encounters Analysis, Operations Analysis, Procedure and Base Cost Analysis, and Patient Returns Analysis Dashboards.

Executive Overview Dashboard

The Executive Overview Dashboard was designed to provide hospital management with a summary of operational activity, financial performance, procedure utilization, and patient behavior. The dashboard consolidates key metrics into a single view, enabling stakeholders to quickly assess overall organizational performance.

Key performance indicators (KPIs) and calculated measures displayed within this dashboard include:

KPI	DAX Measure	Definition
Total Encounters	`Total Encounters = DISTINCTCOUNT('hospital_analytics encounters'[encounter_id])`	The total number of patient encounters recorded during the reporting period.
Total Patients	`Total Patients = DISTINCTCOUNT('hospital_analytics encounters'[patient_id])`	The total number of unique patients recorded within the dataset.
Total Procedures	`Total Procedures = COUNT('hospital_analytics procedures'[patient_id])`	The total number of procedures performed during the reporting period.
Total Claim Cost	`Total Claim Cost = SUM('hospital_analytics encounters'[total_claim_cost])`	The cumulative cost of all patient encounters during the reporting period.
Total Payer Coverage	`Total Payer Coverage = SUM('hospital_analytics encounters'[payer_coverage])`	The total amount contributed by healthcare payers toward claim costs.
Coverage Percentage	`Coverage Percentage = DIVIDE([Total Payer Coverage],[Total Claim Cost])`	The percentage of total claim costs covered by healthcare payers.
Total Procedure Base Cost	`Total Procedure Base Cost = SUM('hospital_analytics procedures'[base_cost])`	The total base cost of all procedures performed during the reporting period.
Average Encounter Duration	`Average Encounter Hours = AVERAGE('hospital_analytics encounters'[duration_hours])`	The average amount of time patients spend in encounters.

Patient Return Rate	`Patient Return Rate = DIVIDE([Returning Patients],[Total Patients])`	The percentage of patients who recorded multiple encounters during the reporting period.

Supporting visualizations were developed to display encounter trends over time, changes in claim costs and payer coverage over time, total encounter by encounter class, and total claim costs by payer membership. These visualizations provide stakeholders with immediate visibility into major trends affecting healthcare operations.

Encounters Analysis Dashboard

The Encounters Analysis Dashboard was developed to provide a deeper analysis of healthcare utilization and encounter-level financial performance.

The dashboard examines:

- Total claim costs and payer coverage by encounter class
- Total coverage by insurance payer
- Average claim cost by encounter class
- Average claim costs by payer membership
- Top 10 encounter descriptions by claim costs

The additional KPIs calculated on this dashboard are as follows:

KPIs	DAX Measure	Definitions
Average Claim Cost	`Average Claim Cost = AVERAGE('hospital_analytics encounters'[total_claim_cost])`	The average claim cost incurred per encounter.
Average Payer Coverage	`Average Payer Coverage = AVERAGE('hospital_analytics encounters'[payer_coverage])`	The average amount contributed by payers per encounter.

The objective of this dashboard is to identify the encounter types that contribute most significantly to healthcare expenditure and operational demand.

Operations Analysis Dashboard

The Operations Analysis Dashboard was created to evaluate healthcare resource utilization and operational efficiency.

Visualizations were developed as follows:

- Total and average encounter base costs by encounter class

- Average encounter duration per day
- Total procedures per day
- Average encounter hours by year
- Average encounter duration by encounter class

Additional KPIs calculated for this dashboard are as follows:

KPIs	DAX Measures	Definitions
Average Encounter Duration	`Average Encounter Hours = AVERAGE('hospital_analytics encounters'[duration_hours])`	The average amount of time patients spend in encounters.
Median Encounter Hours	`Median Encounter Hours = MEDIAN('hospital_analytics encounters'[duration_hours])`	The middle encounter duration value when all encounter durations are arranged in ascending order.
Encounters Longer than 24 Hours	`Encounters Longer than 24 Hours = CALCULATE( COUNTROWS('hospital_analytics encounters'), 'hospital_analytics encounters'[duration_hours] > 24 )`	The number of encounters that lasted more than 24 hours.
Percentage Over 24 Hours	`Percentage Over 24 Hours = DIVIDE( [Encounters Longer than 24 Hours], [Total Encounters] )`	The proportion of total encounters exceeding 24 hours.
Total Encounter Base Cost	`Total Encounter Base Cost = SUM('hospital_analytics encounters'[base_encounter_cost])`	The total operational cost associated with all patient encounters during the reporting period.
Average Encounter Base Cost	`Average Encounter Base Cost = AVERAGE('hospital_analytics encounters'[base_encounter_cost])`	The average operational cost incurred per patient encounter.

The purpose of this dashboard is to identify service areas that place the greatest demand on hospital resources and highlight opportunities for improved operational planning.

Procedures and Base Cost Analysis Dashboard

The Procedures and Base Cost Analysis Dashboard focuses on procedure utilization and procedure-related expenditure.

The dashboard investigates:

- Total procedure base cost by year
- Top 10 average procedure duration
- Top 15 most performed procedures

Additional KPIs calculated for this dashboard are as follows:

KPIs	DAX Measures	Definitions
Total Procedures	`Total Procedures = COUNT('hospital_analytics procedures'[patient_id])`	The total number of procedures performed during the reporting period.
Total Procedure Base Cost	`Total Procedure Base Cost = SUM('hospital_analytics procedures'[base_cost])`	The total base cost of all procedures performed during the reporting period.
Average Procedure Base Cost	`Average Procedure Base Cost = AVERAGE('hospital_analytics procedures'[base_cost])`	The average cost incurred by the hospital to perform a procedure and maintain the resources required to support procedure delivery.
Max Procedure Duration	`Max Procedure Hours = MAX('hospital_analytics procedures'[duration_hours])`	The longest recorded procedure duration within the reporting period.
Average Procedure Hours	`Average Procedure Hours = AVERAGE('hospital_analytics procedures'[duration_hours])`	The average amount of time spent performing procedures during the reporting period.

This dashboard enables stakeholders to identify high-volume procedures, evaluate procedure-related spending, and assess the operational resources required to support healthcare services.

Patient Returns Dashboard

The Patient Returns Dashboard was developed to analyze repeat healthcare utilization and patient return behavior.

The dashboard includes charts for:

- Total patients by county

- Patient return rate by payer membership
- Returning patients and patient return rate by year
- Total patients, returning patients and patient return rate by encounter class

This dashboard provides insight into recurring healthcare utilization patterns.

Dashboard Interactivity

Interactive functionality was incorporated throughout the dashboards to improve usability and support client exploration.

A set of slicers was implemented across all dashboard pages to allow users to filter results consistently. The filters included:

- Date
- Payer
- Encounter Class
- Gender
- Patient Location
- Race

These slicers enabled healthcare stakeholders to explore specific segments of the data while maintaining consistency across all visualizations and KPIs.

4. PROJECT OUTCOMES

This project uncovered a few key insights that will enable management to make data-driven decisions to improve operations and improve payment strategies within the hospital.

- The hospital recorded a total of 27891 patient encounters over a span of 11 years. In those 11 years, a total of approximately $100M in claim costs were recorded. However, payer coverage was only just over $31M, resulting in a coverage rate of 30.6%. This indicates that only a small portion (30.6%) of the hospital bills incurred by patients were covered by insurance payments.
- Almost half (48.5%) of the total claim costs were amassed by patients with no insurance, accounting for $49M out of the total $101.5M in claim costs. Meanwhile, patients who had

Medicare contributed to 24.3% of the total claim costs, suggesting that patients with no insurance accumulated the most costs in hospital bills.

- Patients who are members of Humana accrued $4M in claim costs, yet no payments were made by Humana to cover these costs for the patients. This needs further analysis.
- The total number of procedures performed were 47701. The hospital spent $105.5M to keep beds open, staffed and operational for those procedures.
- With 854 patients out of a 974 total, patient return rate was 87.7%. This implies that patients were visiting the hospital for multiple encounters. The relationship between patient return rate, encounter durations, and base costs requires further investigation.
- Total encounters remained stable throughout the 11 years, but a spike occurred in 2014, with ambulatory encounters accounting for 60.3% of that spike and outpatient encounters accounting for 17.9%. In addition, this spike occurred in January, 2014. Further analysis is required to identify the factors that contributed to this spike in January, 2014.

Conclusion

The Healthcare Intelligence Dashboard successfully transformed complex healthcare data into an interactive reporting solution capable of supporting operational and strategic decision-making. By combining SQL-based analysis with Power BI visualizations, the project provides healthcare stakeholders with comprehensive insights into encounter activity, financial performance, procedure utilization, operational efficiency, and patient behavior.

The resulting dashboard enables users to monitor key performance indicators, identify emerging trends, evaluate financial outcomes, and make more informed decisions based on reliable, data-driven evidence.

5. ISSUES ENCOUNTERED AND TROUBLESHOOTING

Import Issues

MySQL settings were configured to restrict importing CSV files after creating the table. Therefore, the settings were reconfigured using the below query:

```
-- enable local_infile settings to support CSV file import
SHOW variables LIKE 'local_infile';
SET GLOBAL local_infile = 1;
```

After running the query, the CSV files imported correctly and the column headings were validated.

Date Import Issues

Birth dates and death dates within the patient dataset did not import correctly into MySQL during the initial data load. The issue prevented accurate patient age and demographic analysis.

The source CSV date formats were corrected (yyyy-mm-dd), the patient table was recreated, and the data was reloaded into MySQL. Date values were subsequently validated to ensure accuracy.

Payer Coverage Import Issues

Certain base_encounter_cost values that should have been imported as zero were imported as NULL values. Coverage calculations and financial metrics were distorted.

The imported values were validated against the original source files and corrected to preserve the distinction between zero and missing values using the below query:

```
update payers
set base_encounter_cost = 0
where base_encounter_cost ='';
```

Power Query Date Conversion Issues

Valid date values were displayed as 1/1/1 after import into Power Query. Patient date fields could not be used reliably in analysis.

The affected columns were temporarily converted to text format and then reassigned the correct date data type. This forced Power Query to correctly recognize and format the date values.